Cálculo del área de un círculo con diseño modular Top-Down en C++

Alexis Maier

E.E.S.T. N.º 1 «Eduardo Ader», Vicente López

Laboratorio de Programación (LPR) — 5.º año 3.ª división, grupos A-B

Profesor: Mansilla Muñoz York Elías

21 de octubre de 2026

v1.0.0

Cálculo del área de un círculo con diseño modular Top-Down en C++

Introducción

El diseño Top-Down consiste en dividir un problema complejo en módulos más pequeños, de lo general a lo particular. En C++ esto se logra con funciones creadas por el programador, lo que facilita la depuración, la reutilización del código y el trabajo en equipo.

En esta actividad se desarrolló un programa modular que calcula el área de un círculo con la fórmula $A = \pi \cdot r^2$, validando el dato ingresado y simulando una diana ASCII como bonus de gamificación. El informe explica la estructura del programa, muestra la evidencia de ejecución y responde el cuestionario técnico.

Desarrollo

Diseño del programa

El programa se divide en cuatro bloques: (1) el prototipo `double calcularAreaCirculo(double radio);` antes de `main`, (2) el filtro de consistencia que rechaza radios menores o iguales a cero, (3) la llamada a la función guardando el resultado en `areaFinal` y su impresión, y (4) la definición de la función, que declara `const double PI = 3.1415926535;` y retorna `PI r r`.

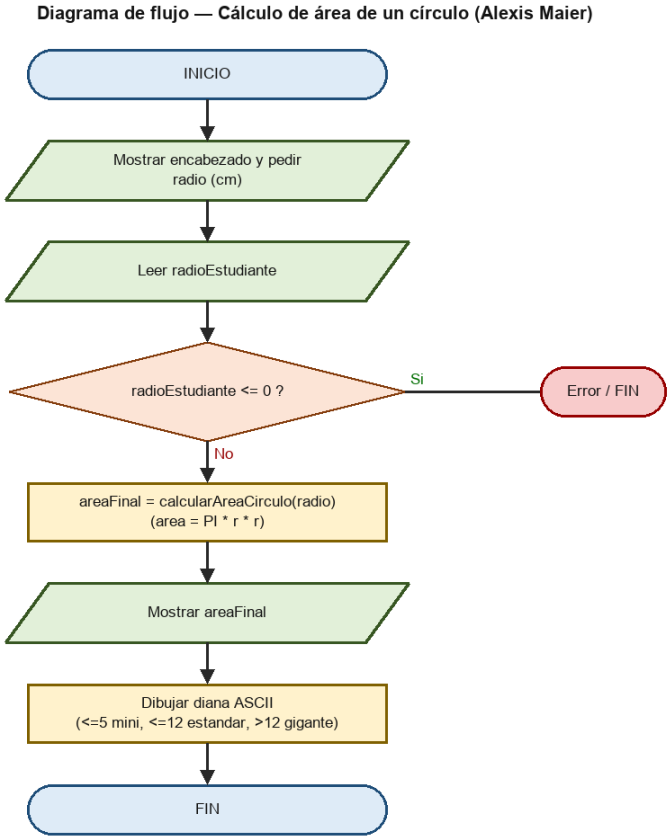


Figura 1. Diagrama de flujo del programa.

Partes completadas de la plantilla

```
// Llamada / invocación (bloque 2)
double areaFinal = calcularAreaCirculo(radioEstudiante);

// Salida de datos (bloque 3)
cout << "\n[RESULTADO] El area del circulo es: " << areaFinal << " cm2" << endl;

// Definición de la función (bloque 4)
double calcularAreaCirculo(double r) {
   const double PI = 3.1415926535;
   double area = PI * r * r;
   return area;
}
```

Compilación y ejecución

Se compiló con `g++ src/main.cpp -o src/area.exe` y se probó con cuatro radios: 8 cm (diana estándar, área ≈ 201,06 cm²), 3 cm (diana mini, área ≈ 28,27 cm²), 15 cm (diana gigante, área ≈ 706,86 cm²) y −5 cm (rechazado por el filtro).

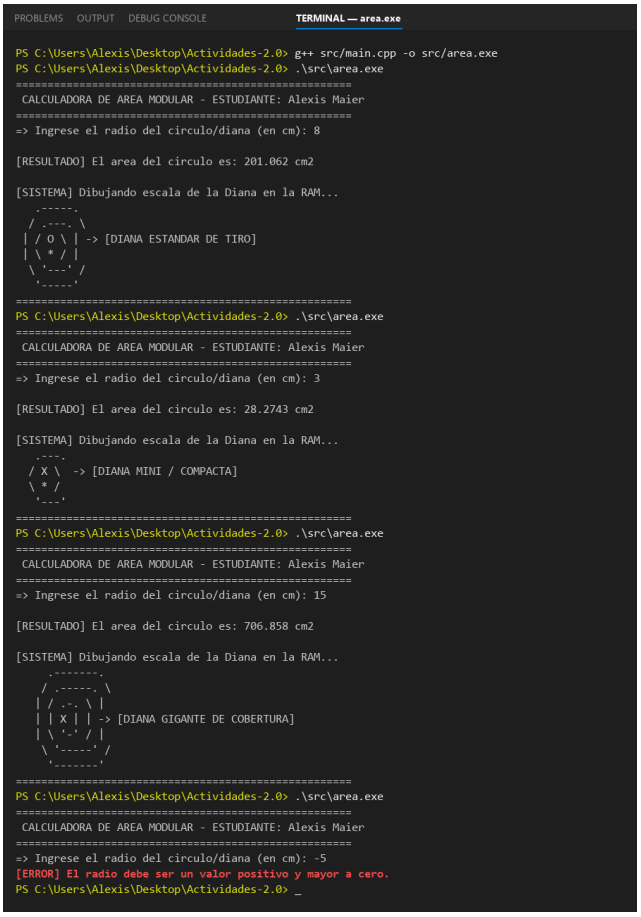


Figura 2. Compilación y ejecuciones del programa con distintos radios.

Cuestionario técnico de control

Pregunta 1: El prototipo de la función. Si borráramos la línea del prototipo, ¿por qué el compilador no sabría qué hacer al leer main?

El compilador lee el archivo de arriba hacia abajo. Al llegar a la llamada `calcularAreaCirculo(radioEstudiante)` dentro de `main`, la función todavía no fue declarada ni definida (su desarrollo está al final del archivo), de modo que el compilador no conoce su nombre, el tipo de dato que devuelve ni los parámetros que recibe. Por eso lanza un error del tipo «'calcularAreaCirculo' was not declared in this scope». El prototipo adelanta esa «firma» y permite dejar el desarrollo de la función después de `main`.

Pregunta 2: El filtro de consistencia. ¿Qué problema físico o geométrico ocurriría con un radio de 0 cm o de −5 cm? ¿Por qué es importante controlarlo?

Un radio de 0 cm representa un punto sin superficie: la fórmula daría área 0, un resultado sin sentido para una diana o un círculo real. Un radio de −5 cm no existe geométricamente (una longitud no puede ser negativa); como el radio se eleva al cuadrado,

la fórmula devolvería 78,54 cm² igual que para +5 cm y ocultaría el error del usuario. Controlarlo evita resultados falsos que se propagan al resto del sistema, que es lo que en el mundo real genera fallas silenciosas.

Pregunta 3: Proteger constantes en RAM. ¿Por qué es más seguro declarar valores como inmutables (const) en un sistema crítico?

Porque el compilador bloquea cualquier intento de modificar el valor: si un programador escribe por error `PI = 5.5;`, la compilación se detiene. En un software de avión o un equipo médico, un valor como una constante física o un límite de seguridad alterado accidentalmente por otra parte del programa podría causar cálculos erróneos con consecuencias graves. `const` convierte un error en tiempo de ejecución, difícil de detectar, en un error de compilación, que se corrige antes de que el programa llegue al usuario.

Conclusión

El enfoque modular permitió separar la validación, el cálculo y la presentación del resultado, de modo que cada parte puede probarse y modificarse de forma independiente. El prototipo, el filtro de consistencia y la constante `const` son tres mecanismos simples que vuelven el programa más robusto y legible, y son la base para estructurar programas más grandes.

Referencias

cplusplus.com. (s. f.). *Functions*. https://cplusplus.com/doc/tutorial/functions/

Microsoft. (s. f.). *Referencia del lenguaje C++*. https://learn.microsoft.com/es-es/cpp/cpp/cpp-language-reference

Mansilla Muñoz, Y. E. (2026). *Actividad 3: Enfoque modular y diseño Top-Down* [Guía de actividad]. E.E.S.T. N.º 1 «Eduardo Ader».